

Lab 9

Example 1

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t mutex;

void* thread(void* arg)
{
    // wait
    sem_wait(&mutex);

    // critical section
    printf("Thread entered critical section\n");
    sleep(4);
    printf("Just Exiting...\n");

    // signal
    sem_post(&mutex);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    sem_init(&mutex, 0, 1);

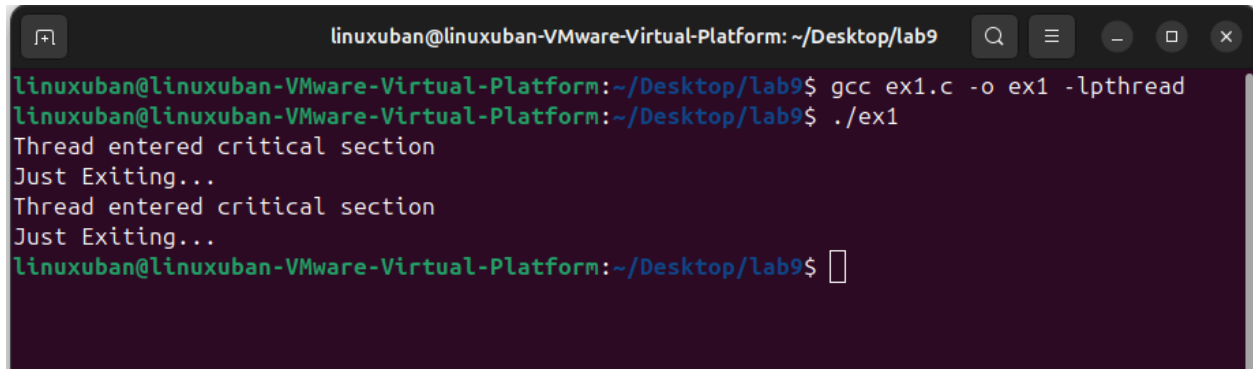
    pthread_create(&t1, NULL, thread, NULL);
    sleep(2);
    pthread_create(&t2, NULL, thread, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    sem_destroy(&mutex);

    return 0;
}
```

}

A terminal window titled 'linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab9'. The window shows the compilation and execution of a C program. The user enters 'gcc ex1.c -o ex1 -lpthread' to compile the program. Then, they enter './ex1' to run it. The output shows two threads entering and exiting a critical section. The terminal text is as follows:

```
linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab9$ gcc ex1.c -o ex1 -lpthread
linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab9$ ./ex1
Thread entered critical section
Just Exiting...
Thread entered critical section
Just Exiting...
linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab9$
```

Example 2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <semaphore.h>

int main()
{
    int i;
    int shmid_data, shmid_sem;
    int *counter;
    sem_t *mutex;

    /* Shared memory for integer */
    shmid_data = shmget(IPC_PRIVATE, sizeof(int), 0666);
    counter = (int *) shmat(shmid_data, NULL, 0);
    *counter = 0;
```

```

/* Shared memory for semaphore */
shmid_sem = shmget(IPC_PRIVATE, sizeof(sem_t), 0666);
mutex = (sem_t *) shmat(shmid_sem, NULL, 0);

sem_init(&mutex, 1, 1); // shared between processes

if (fork() == 0)
{
    sem_wait(&mutex);

    for (i = 0; i < 5; i++)
        printf("Child: %d\n", (*counter)++);

    sem_post(&mutex);
    exit(0);
}

wait(NULL);

sem_wait(&mutex);

for (i = 0; i < 5; i++)
    printf("Parent: %d\n", (*counter)++);

sem_post(&mutex);

sem_destroy(&mutex);

shmctl(shmid_data, IPC_RMID, NULL);
shmctl(shmid_sem, IPC_RMID, NULL);

return 0;
}

```

```
linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab9
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab9$ gcc ex1.c -o ex1 -lpthread
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab9$ ./ex1
Child: 0
Child: 1
Child: 2
Child: 3
Child: 4
Parent: 5
Parent: 6
Parent: 7
Parent: 8
Parent: 9
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab9$
```

Q1

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int balance = 1000; // shared bank balance
sem_t mutex;

// Deposit thread
void* deposit(void* arg)
{
    int amount = 500;
    sem_wait(&mutex); // lock

    printf("Depositing money...\n");
    balance += amount;
    printf("Updated Balance after deposit: %d\n", balance);

    sem_post(&mutex); // unlock
    return NULL;
}

// Withdraw thread
void* withdraw(void* arg)
{
```

```

    int amount = 300;
    sem_wait(&mutex); // lock

    printf("Withdrawing money...\n");
    balance -= amount;
    printf("Updated Balance after withdrawal: %d\n", balance);

    sem_post(&mutex); // unlock
    return NULL;
}

int main()
{
    pthread_t t1, t2;

    sem_init(&mutex, 0, 1); // binary semaphore

    pthread_create(&t1, NULL, deposit, NULL);
    pthread_create(&t2, NULL, withdraw, NULL);

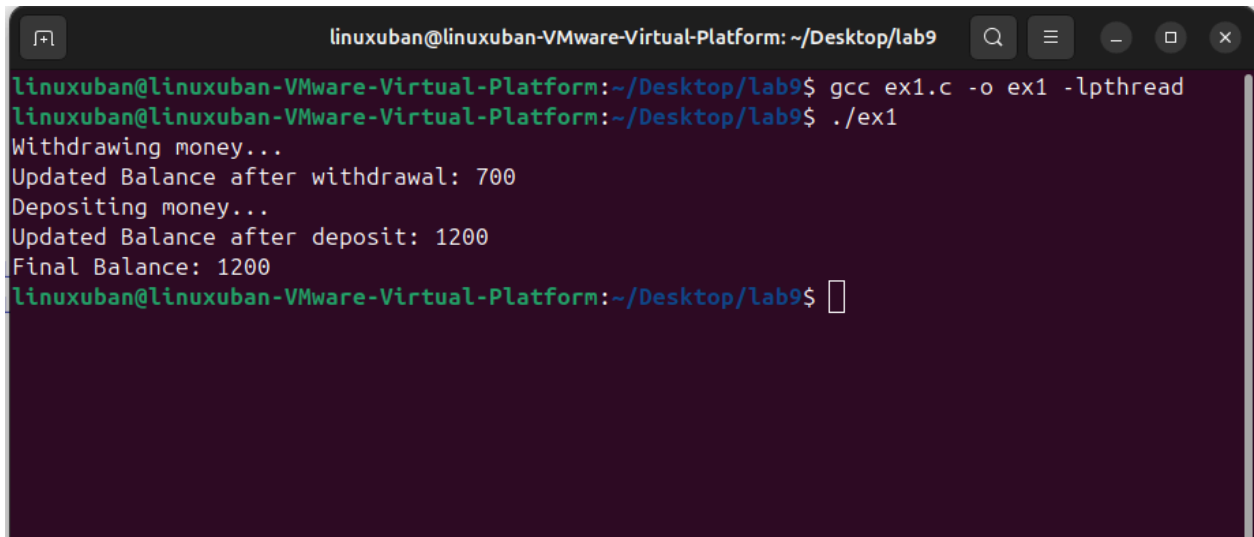
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    sem_destroy(&mutex);

    printf("Final Balance: %d\n", balance);

    return 0;
}

```



```

linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab9
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab9$ gcc ex1.c -o ex1 -lpthread
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab9$ ./ex1
Withdrawing money...
Updated Balance after withdrawal: 700
Depositing money...
Updated Balance after deposit: 1200
Final Balance: 1200
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab9$ 

```

Q2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <semaphore.h>

int main()
{
    int shmid_sem;
    sem_t *printer_mutex;

    /* Shared memory for semaphore */
    shmid_sem = shmget(IPC_PRIVATE, sizeof(sem_t), 0666);
    printer_mutex = (sem_t *) shmat(shmid_sem, NULL, 0);

    /* Initialize semaphore for process sharing */
    sem_init(printer_mutex, 1, 1); // pshared = 1 (between processes)

    if (fork() == 0)
    {
        // Child process
        sem_wait(printer_mutex);

        printf("Process 1: Sending print job...\n");
        sleep(3);
        printf("Process 1: Print job completed\n");

        sem_post(printer_mutex);
        exit(0);
    }

    if (fork() == 0)
    {
        // Second child process
        sem_wait(printer_mutex);

        printf("Process 2: Sending print job...\n");
        sleep(3);
```

```

        printf("Process 2: Print job completed\n");

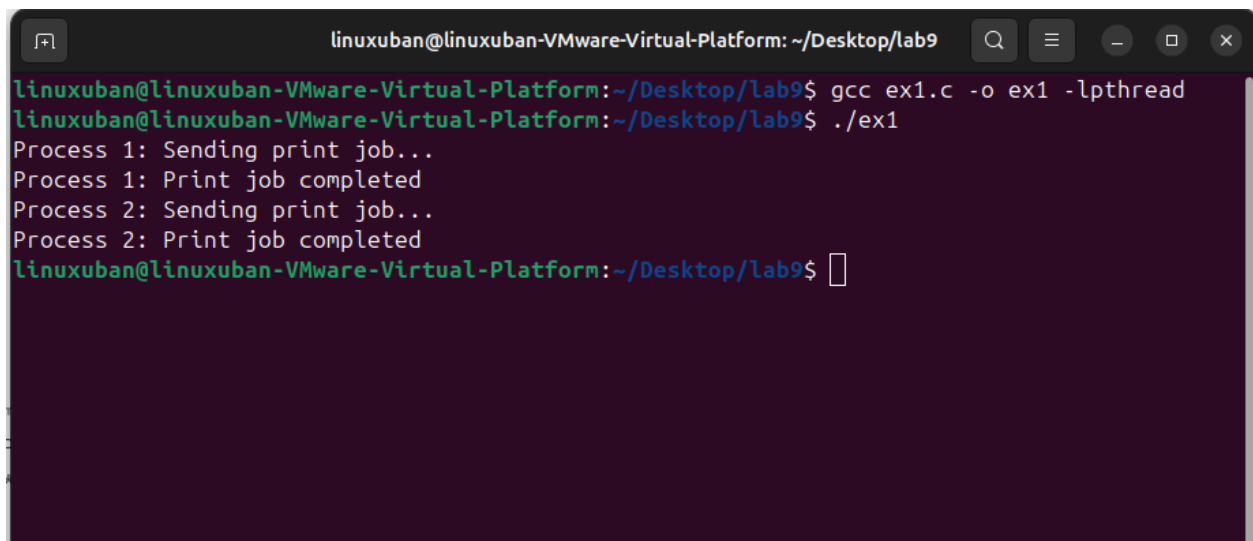
        sem_post(printer_mutex);
        exit(0);
    }

    // Parent waits for both children
    wait(NULL);
    wait(NULL);

    /* Cleanup */
    sem_destroy(printer_mutex);
    shmctl(shmid_sem, IPC_RMID, NULL);

    return 0;
}

```



A terminal window titled 'linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab9' shows the compilation and execution of a C program. The user runs 'gcc ex1.c -o ex1 -lpthread' and then './ex1'. The output shows two parallel processes, Process 1 and Process 2, each sending a print job and then printing 'Print job completed'.

```

linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab9$ gcc ex1.c -o ex1 -lpthread
linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab9$ ./ex1
Process 1: Sending print job...
Process 1: Print job completed
Process 2: Sending print job...
Process 2: Print job completed
linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab9$ 

```

Q3

```

#include <iostream>
#include <pthread.h>
#include <semaphore.h>

using namespace std;

int tickets = 10;    // Shared resource

```

```

sem_t mutex;          // Binary semaphore

void* bookTicket(void* arg)
{
    int id = *(int*)arg;

    sem_wait(&mutex); // Enter critical section

    if (tickets > 0)
    {
        cout << "Counter " << id << " booked 1 ticket" << endl;
        tickets--;
        cout << "Remaining tickets: " << tickets << endl;
    }
    else
    {
        cout << "Counter " << id << " found no tickets" << endl;
    }

    sem_post(&mutex); // Exit critical section

    pthread_exit(NULL);
}

int main()
{
    pthread_t t1, t2, t3, t4;

    sem_init(&mutex, 0, 1); // Initialize binary semaphore

    int id1 = 1, id2 = 2, id3 = 3, id4 = 4;

    pthread_create(&t1, NULL, bookTicket, &id1);
    pthread_create(&t2, NULL, bookTicket, &id2);
    pthread_create(&t3, NULL, bookTicket, &id3);
    pthread_create(&t4, NULL, bookTicket, &id4);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    sem_destroy(&mutex);
}

```



```
    return 0;  
}
```

g++ q3.cpp -o q3 -lpthread

```
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab9$ ./q3  
Counter 1 booked 1 ticket  
Remaining tickets: 9  
Counter 2 booked 1 ticket  
Remaining tickets: 8  
Counter 3 booked 1 ticket  
Remaining tickets: 7  
Counter 4 booked 1 ticket  
Remaining tickets: 6  
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab9$
```